

DeporData — Documento de arquitectura

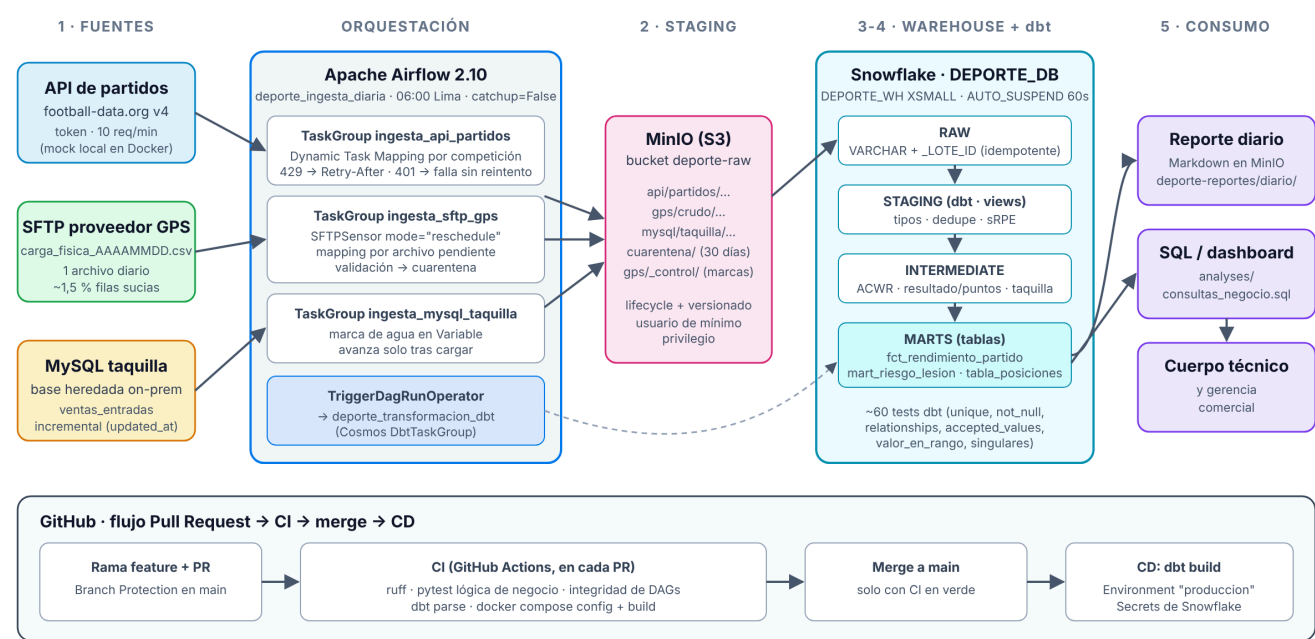
Proyecto Final Integrador · PEDE/9 Apache Airflow · Prof. Renato Arrascue

Equipo: (nombres) · Repositorio: (link público) · Video demo: (link)

1. Caso de negocio

El **Club Deportivo Andino** necesita responder cada mañana tres preguntas: **¿cómo nos fue?**, **¿en qué estado llega el plantel?** y **¿cuánto recaudamos?** Los datos viven en tres sistemas aislados: la **API de resultados** (football-data.org), los **CSV de los chalecos GPS** que el proveedor deja en un **SFTP**, y la **base MySQL heredada de taquilla**. DeporData los integra a diario y entrega al cuerpo técnico un semáforo de **riesgo de lesión por jugador (ACWR)** y a la gerencia una tabla de **rendimiento por partido** que cruza resultado, carga física previa y recaudación.

2. Arquitectura



- Ingesta (Airflow 2.10, deporte_ingesta_diaria, 06:00 Lima):** un TaskGroup por fuente. API con Dynamic Task Mapping por competición; SFTP con SFTPSensor en mode="reschedule" y mapping por archivo pendiente; MySQL con extracción incremental por marca de agua.
- Staging intermedio (MinIO):** todo dato crudo queda en deporte-raw (particionado por fecha) antes de Snowflake; las filas inválidas, en cuarentena/. Lifecycle policies y usuario de mínimo privilegio.
- Carga (Snowflake RAW):** tablas VARCHAR + columnas de linaje (_LOTE_ID, _CARGADO_EN), carga idempotente borrar-e-insertar por lote.
- Transformación (dbt + Cosmos, deporte_transformacion_dbt):** 10 modelos en staging → intermediate → marts y ~60 tests, cada modelo como tarea de Airflow (DbtTaskGroup).
- Consumo:** fct_rendimiento_partido, mart_riesgo_lesion, mart_tabla_posiciones, reporte diario en MinIO y consultas SQL de ejemplo.

CI/CD: PR → GitHub Actions (ruff, pytest, integridad de DAGs, dbt parse, build de Docker) → Branch Protection → merge → CD (dbt build con el Environment produccion y sus Secrets).

3. Decisiones de diseño

Decisión	Por qué
Tres fuentes de tipos distintos	Cubren los patrones del curso: API con límite de tasa, archivo por SFTP y base heredada on-premise.

Decisión	Por qué
Mock fiel de la API	Contrato idéntico a football-data.org (token, 429, rango de 10 días): el proyecto corre en cualquier máquina y se pasa a la API real cambiando solo la Connection.
Errores tipados (definitivos vs reintentables)	Un token inválido falla al instante sin gastar reintentos; un 429 o 5xx se reintenta con backoff exponencial. Nada falla en silencio.
Cuarentena + umbral de calidad	Unas pocas filas sucias no deben bloquear el día; muchas indican un cambio de formato y detienen ese archivo.
"Procesar todos los pendientes"	Si un día falla o el proveedor llega tarde, la siguiente corrida se pone al día sola; la primera corrida hace el backfill.
Batch incremental (no CDC)	Una carga diaria basta y no exige permisos de replicación en un sistema heredado. La marca de agua avanza solo tras cargar (al-menos-una-vez + dedupe en dbt).
RAW en VARCHAR	Un cambio de formato en la fuente no rompe la carga: lo detecta un test de dbt.
dbt corre aunque falle una fuente	Modelos idempotentes; el fallo se hace visible con <code>verificar_corrida</code> y la alerta.
On-prem vs nube	La taquilla sigue on-premise (costo de migrarla); el warehouse va a la nube por elasticidad (XSMALL, <code>AUTO_SUSPEND 60</code> , monitor de 5 créditos/mes) y Time Travel. Airflow podría migrar a MWAA/Composer/Astronomer sin cambios de código.

Tests de dbt priorizados. Primero los que protegen el indicador más sensible —el riesgo de lesión—: `relationships` del jugador GPS contra el plantel, rango del RPE (1–10), unicidad jugador-día de la serie y rango del ACWR (`warn`). Luego las reglas de negocio: puntos coherentes con el marcador, ventas que no superan el aforo de cada tribuna y tabla de posiciones que cuadra (goles a favor = en contra). Por último, la integridad clásica (`unique/not_null`) de las claves de cada modelo.

Seguridad. Cero credenciales en Git (`.env` ignorado, Connections `AIRFLOW_CONN_*`, `env_var()` en dbt, test de CI que busca secretos). Mínimo privilegio en MySQL (solo `SELECT`), MinIO (sin borrar ni crear buckets) y Snowflake (rol propio acotado a `DEPORTE_DB`).